Regression under Covariate Shift: From ERM to Importance Weighting

Moutie Mohamed Ali, Bleich Duncan, Olszewska Urszula
Statistical Machine Learning Project, EPFL

Abstract. When training and test inputs follow different distributions, empirical risk minimization (ERM) estimates the wrong risk and can yield poor test performance. Under *covariate shift*, the conditional distribution $p(y \mid x)$ is unchanged but the marginal $p(x)$ differs. We explain why this breaks ERM, how importance sampling yields a corrected objective, and how importance-weighted ERM (IWERM) implements this correction in practice. Since the required importance weights are unknown, we use density ratio estimation (DRE) via probabilistic classification. Finally, we show that model selection must also be shift-aware: standard cross-validation tunes hyperparameters for the training distribution, whereas importance-weighted cross-validation (IWCV) better targets the test risk. A controlled 1D experiment illustrates these effects.

1 Covariate Shift and Why ERM Fails

We consider supervised regression on $\mathcal{X} \subset \mathbb{R}^d$, $\mathcal{Y} \subset \mathbb{R}$ with $y = f^*(x)+\varepsilon$, where f^* is unknown and ε is noise. We observe training pairs $(x_i^{\mathrm{tr}}, y_i^{\mathrm{tr}})_{i=1}^{n_{\mathrm{tr}}} \sim p_{\mathrm{tr}}(x)\, p(y \mid x)$, and evaluate on test pairs $(x^{\mathrm{te}}, y^{\mathrm{te}}) \sim p_{\mathrm{te}}(x)\, p(y \mid x)$. The goal is to minimize the *test risk* $R_{\mathrm{te}}(f) = \mathbb{E}_{x\sim p_{\mathrm{te}},\, y|x}\big[\ell(f(x), y)\big]$, $\ell(\hat{y}, y) = (\hat{y} - y)^2$. ERM instead minimizes the empirical training risk

$$\widehat{R}_{\mathrm{tr}}(f) = \frac{1}{n_{\mathrm{tr}}} \sum_{i=1}^{n_{\mathrm{tr}}} \ell(f(x_i^{\mathrm{tr}}), y_i^{\mathrm{tr}}),$$

which targets the population quantity $R_{\mathrm{tr}}(f) = \mathbb{E}_{x\sim p_{\mathrm{tr}},\, y|x}[\ell(f(x), y)]$. When $p_{\mathrm{tr}}(x) = p_{\mathrm{te}}(x)$, this is aligned with the test objective. Under **covariate shift**, however,

$$p_{\mathrm{tr}}(x) \neq p_{\mathrm{te}}(x) \quad \text{while} \quad p_{\mathrm{tr}}(y \mid x) = p_{\mathrm{te}}(y \mid x).$$

Then ERM prioritizes accuracy where training inputs are frequent, even if those regions matter little at test time; the minimizers of R_{tr} and R_{te} can differ substantially, especially with a misspecified model class.

Running 1D Example We consider throughout the empirical part of this report the following controlled 1D setting:

$$f^*(x) = \sin(2\pi x), \quad x^{\mathrm{tr}} \sim \mathcal{N}(0.75, 0.3^2),$$
$$x^{\mathrm{te}} \sim \mathcal{N}(1.25, 0.1^2), \quad \varepsilon \sim \mathcal{N}(0, 0.4^2).$$

This example exhibits a clear covariate shift: training inputs concentrate near $x \approx 0.75$, while test inputs concentrate near $x \approx 1.25$. With a linear model class, ERM fits the training region but can extrapolate poorly in the test region, as can be seen in Figure 1.

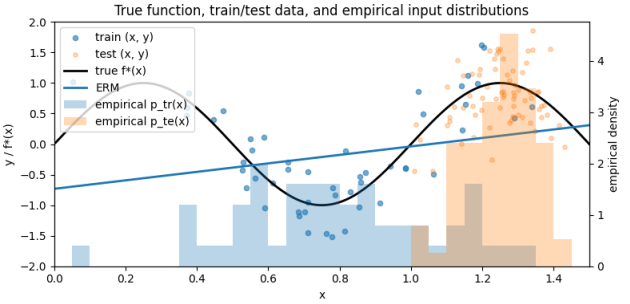


Figure 1: Covariate shift can break ERM: minimizing training risk does not guarantee low test risk.

2 Fixing the Objective with Importance Weighting

Importance sampling allows us to rewrite expectations under p_{te} using samples from p_{tr}. For any integrable g,

$$\mathbb{E}_{x\sim p_{\mathrm{te}}}[g(x)] = \mathbb{E}_{x\sim p_{\mathrm{tr}}}\left[\frac{p_{\mathrm{te}}(x)}{p_{\mathrm{tr}}(x)}\, g(x)\right].$$

Applying this to the loss yields

$$R_{\mathrm{te}}(f) = \mathbb{E}_{x\sim p_{\mathrm{tr}},\, y|x}[w(x)\, \ell(f(x), y)], \qquad w(x) = \frac{p_{\mathrm{te}}(x)}{p_{\mathrm{tr}}(x)}.$$

The weights have a direct meaning: $w(x)$ is large when x is test-typical but training-rare, so errors there should influence learning more; $w(x)$ is small when x is training-typical but test-rare, so errors there should matter less for target performance.

This motivates importance-weighted ERM:

$$\widehat{R}_{\mathrm{IW}}(f) = \frac{1}{n_{\mathrm{tr}}} \sum_{i=1}^{n_{\mathrm{tr}}} w(x_i^{\mathrm{tr}})\, \ell(f(x_i^{\mathrm{tr}}), y_i^{\mathrm{tr}}),$$
$$\widehat{f}_{\mathrm{IWERM}} \in \arg\min_{f\in\mathcal{F}} \widehat{R}_{\mathrm{IW}}(f).$$

If $w(x) = p_{\mathrm{te}}(x)/p_{\mathrm{tr}}(x)$ were known, $\widehat{R}_{\mathrm{IW}}$ would estimate the test risk using only training data. In practice, w must be estimated from all samples, which leads to density ratio estimation.

3 Density-Ratio Estimation

To apply IWERM, we need the importance weights

$$w(x) = \frac{p_{\mathrm{te}}(x)}{p_{\mathrm{tr}}(x)},$$

but only samples from p_{tr} and p_{te} are available. Density Ratio Estimation (DRE) provides methods to estimate $w(x)$ directly from unlabeled data.

The main idea is to learn a probabilistic classifier that separates training samples $\{x_i^{\text{tr}}\}$ and test samples $\{x_i^{\text{te}}\}$. First, we assign labels $y = +1$ to $\{x_i^{\text{tr}}\}$ and $y = -1$ to $\{x_i^{\text{te}}\}$, respectively. Then the two densities $p^*_{\text{tr}}(x)$ and $p^*_{\text{te}}(x)$ are written as

$$p^*_{\text{tr}}(x) = p^*(x|y = +1) \quad \text{and} \quad p^*_{\text{te}}(x) = p^*(x|y = -1),$$

respectively. Note that y is regarded as a random variable here. If we now apply Bayes' theorem,

$$p^*(x|y) = \frac{p^*(y|x)p^*(x)}{p^*(y)},$$

we get that the density ratio, say $r^*(x)$, can be expressed in terms of y as

$$\begin{aligned} r^*(x) &= \frac{p^*_{\text{tr}}(x)}{p^*_{\text{te}}(x)} \\ &= \left(\frac{p^*(y=+1|x)p^*(x)}{p^*(y=+1)}\right)\left(\frac{p^*(y=-1|x)p^*(x)}{p^*(y=-1)}\right)^{-1} \\ &= \frac{p^*(y=-1)}{p^*(y=+1)}\frac{p^*(y=+1|x)}{p^*(y=-1|x)}. \end{aligned}$$

The prior ratio $p^*(y=-1)/p^*(y=+1)$ may be simply approximated by the ratio of the sample size

$$\frac{p^*(y=-1)}{p^*(y=+1)} \approx \frac{n_{\text{te}}/(n_{\text{tr}}+n_{\text{te}})}{n_{\text{tr}}/(n_{\text{tr}}+n_{\text{te}})} = \frac{n_{\text{te}}}{n_{\text{tr}}},$$

where n_{tr}, n_{te} are training and testing sample size respectively. The posterior probability $p^*(y|x)$ may be approximated by separating $\{x_i^{\text{tr}}\}$ and $\{x_i^{\text{te}}\}$ using a probabilistic classifier. Thus, given an estimator of the class-posterior probability, $\hat{p}(y|x)$, a density-ratio estimator $\hat{r}(x)$ can be constructed as

$$\hat{r}(x) = \frac{n_{\text{te}}}{n_{\text{tr}}}\frac{\hat{p}(y=+1|x)}{\hat{p}(y=-1|x)}.$$

We now take closer look at two of the techniques for density ratio estimation methods based on logistic regression and the least-squares probabilistic classifier. All of these methods are based on the powerful idea presented above.

Logistic regression Let $y \in \{-1, +1\}$. We model the class-posterior probability as

$$p(y \mid x; w) = \sigma\big(y\, w^\top \psi(x)\big) = \frac{1}{1+\exp(-y\, w^\top \psi(x))},$$

where $\psi(x) : \mathbb{R}^d \to \mathbb{R}^b$ is a vector of basis functions and $w \in \mathbb{R}^b$ is the parameter vector. The basis function $\psi(x)$ allows the classifier to model nonlinear decision boundaries in the original input space while remaining linear in the parameter vector w. So a density-ratio estimator $\widehat{r}_{\text{LR}}(x)$ is given by

$$\begin{aligned} \widehat{r}_{\text{LR}}(x) &= \frac{n_{\text{te}}}{n_{\text{tr}}}\frac{\sigma\big(\widehat{w}^\top\psi(x)\big)}{\sigma(-\widehat{w}^\top\psi(x))} = \frac{n_{\text{te}}}{n_{\text{tr}}}\frac{1+\exp\big(\widehat{w}^\top\psi(x)\big)}{1+\exp\left(-\widehat{w}^\top\psi(x)\right)} \\ &= \frac{n_{\text{te}}}{n_{\text{tr}}}\frac{\exp\big(\widehat{w}^\top\psi(x)\big)\left(\exp\left(-\widehat{w}^\top\psi(x)\right)+1\right)}{1+\exp\left(-\widehat{w}^\top\psi(x)\right)} \\ &= \frac{n_{\text{te}}}{n_{\text{tr}}}\exp\big(\widehat{w}^\top\psi(x)\big), \end{aligned}$$

The parameter vector w is learned so that the penalized log-likelihood is maximized, which can be expressed as the following minimization problem:

$$\widehat{w} := \underset{w\in\mathbb{R}^b}{\operatorname{argmin}}\left[\sum_{k=1}^{n}\log\left(1+\exp\left(-\gamma_k\psi(x_k)^\top w\right)\right)+\lambda||w||_2^2\right],$$

where $\lambda||w||_2^2$ is an ℓ_2-regularization term and $\gamma_k \in \{-1, +1\}$ denotes the label of the sample x_k. The regularization parameter $\lambda > 0$ controls the trade-off between fitting the training data and maintaining model simplicity, thereby enhancing generalization performance.

As described above, now applied to our 1D running example, we estimate the density ratio by first casting the problem as a binary classification task between samples drawn from the training and test distributions. To capture potential nonlinear structure in the data, we employ kernel logistic regression, where the inputs are mapped to a high-dimensional feature space induced by a radial basis function (RBF) kernel. Since an exact kernel representation is computationally expensive, we approximate this feature map using the Nystrm method, yielding a finite-dimensional representation in which standard ℓ_2-regularized logistic regression can be efficiently trained. The resulting classifier then provides an estimate of the density ratio through the relation between class probabilities and likelihood ratios.

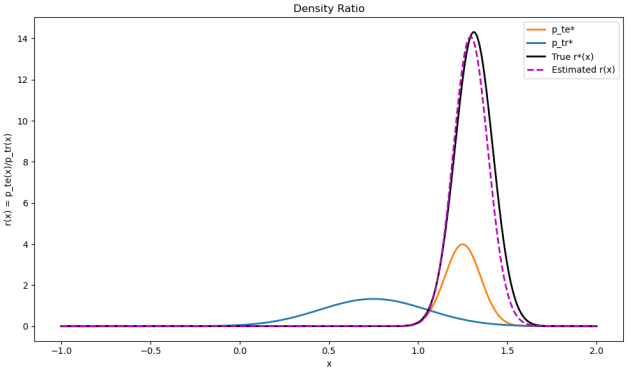


Figure 2: Estimated and actual density ratio for our example as well as the distributions of training and testing data.

Observing that the above classification-based approach to density ratio estimation can be implemented using different probabilistic classifiers to distinguish training and test samples treated as two classes, we briefly introduce another widely used method: the Least-Squares Probabilistic Classifier (LSPC). LSPC is generally more computationally efficient than logistic regression while often providing comparable empirical performance. In LSPC, the class-posterior probability $p(y|x)$ is modeled as

$$p(y|x;w) := \sum_{\ell=1}^{b} w_\ell \psi(x,y) = \psi(x,y)^\top w,$$

where $\psi(x,y) \in \mathbb{R}^b$ is a non-negative basis function vector and $w \in \mathbb{R}^b$ is a parameter vector. The class label y takes a value in $\{1,\ldots,c\}$, where c denotes the number of classes. To ensure that LSPC produces a valid probability, the outputs are normalized and negative values are clipped to zero. The LSPC solution can be computed analytically by solving a regularized system of linear equations; consequently, its computation is highly efficient and numerically stable.

Figure 3: Estimated importance weights $\hat{w}(x)$ corresponding to our data; the weights increase as we move toward the testing region, as desired.

Importance weights. Figure 3 shows the estimated weights $\hat{w}(x)$ from logistic DRE. The weights are small near the training region and increase as we move toward the testing region, reflecting that points typical of p_{te} but rare under p_{tr} should have greater influence in IWERM. The wide range of weights (mean 0.592, min = 0.047, max = 4.257) contributes to the instability of unregularized IWERM and is a driver for the need for regularization, which we revisit and analyze in detail in the subsequent sections.

4 Experiment and Shift-Aware Model Selection

Setup. We recall the 1D experiment setup mentioned previously and precise the model class considered:

$$f^*(x) = \sin(2\pi x), \quad x^{\text{tr}} \sim \mathcal{N}(0.75, 0.3^2),$$
$$x^{\text{te}} \sim \mathcal{N}(1.25, 0.1^2), \quad \varepsilon \sim \mathcal{N}(0, 0.4^2).$$

We consider a polynomial regression model of the form

$$f_\theta(x) = \sum_{j=0}^{M} \theta_j x^j,$$

where M denotes the polynomial degree. In the first part of the experiments we set $M = 1$ (linear regression), and we later will consider a more flexible case with $M = 10$.

ERM vs IWERM. With a misspecified model (degree-1 polynomial), ERM fits primarily the training-dense region and therefore extrapolates poorly toward the test region. In our experiment ($n_{\text{tr}} = 50$, $n_{\text{te}} = 75$), this results in a train MSE of 0.6275 and a higher test MSE of 0.6772.

IWERM instead reweights the training samples using the estimated importance weights $\hat{w}(x)$, giving more influence to points that are test-typical but training-rare. As visible in Figure 4, this shifts the fitted line toward the test region and leads to a noticeably better match with the test data. Quantitatively, IWERM reduces the test MSE to 0.2666, while also achieving a lower importance-weighted training MSE of 0.3783 relative to ERM. We emphasize that this simultaneous improvement on the training (importance-weighted) objective is specific to this particular example, the shape of f^*, and the relative placement of p_{tr} and p_{te}; in general, IWERM is not guaranteed-and technically should not be expected-to improve performance on the training distribution.

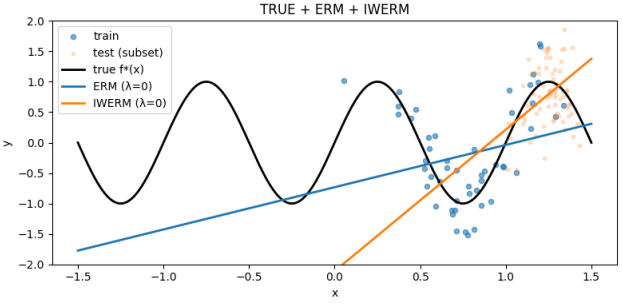


Figure 4: ERM vs IWERM (degree 1). The improvement comes from matching the learning objective to the test distribution.

Why regularization matters. Importance weighting can increase variance: a small number of training points may receive large weights (test-typical but

training-rare), and the weighted objective may become dominated by noisy samples. For flexible models, this can cause severe overfitting. We therefore consider regularized IWERM (ridge):

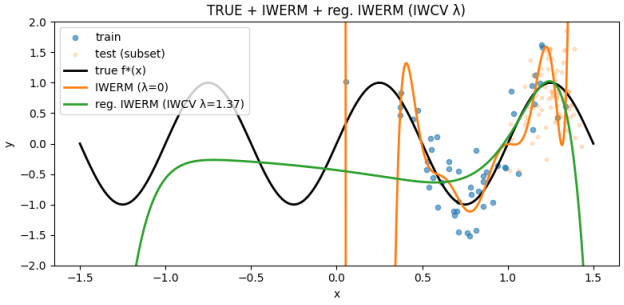
$$\hat{\theta}_\lambda = \arg\min_\theta \left[\frac{1}{n_{\text{tr}}} \sum_{i=1}^{n_{\text{tr}}} w(x_i^{\text{tr}}) \left(f_\theta(x_i^{\text{tr}}) - y_i^{\text{tr}} \right)^2 + \lambda \|\theta\|_2^2 \right].$$

Tuning of the hyperparameter λ and why standard CV fails under shift. Standard K-fold CV evaluates validation error on folds drawn from p_{tr}, so it selects λ that is optimal for the *training* distribution. Under covariate shift, the target is performance under p_{te}, so the chosen λ can be systematically biased. Often too small, preferring overly flexible fits that perform poorly in the test region.

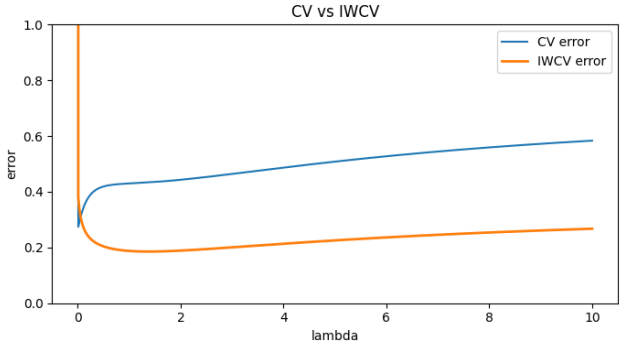
IWCV. Importance-weighted cross-validation corrects the validation criterion by reweighting the validation loss:

$$\text{IWCV}(\lambda) = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{|V_k|} \sum_{i \in V_k} w(x_i^{\text{tr}}) \left(f_{\theta_\lambda^{(-k)}}(x_i^{\text{tr}}) - y_i^{\text{tr}} \right)^2.$$

This approximates a test-risk criterion while using only training labels and estimated weights.



(a) Degree-10: IWERM vs regularized IWERM.



(b) CV vs IWCV tuning curves.

Figure 5: Panel (a) compares unregularized IWERM with IWERM tuned by 5-fold IWCV. Panel (b) shows 5-fold validation curves for λ: blue is standard CV on p_{tr}, orange is IWCV using the estimated weights $\hat{w}(x) = \hat{p}_{\text{te}}(x)/\hat{p}_{\text{tr}}(x)$.

Effect of regularization and shift-aware model selection. Figure 5 illustrates both the impact of regularization under importance weighting and the difference between standard CV and IWCV for selecting λ. In panel (a), the orange curve corresponds to unregularized IWERM ($\lambda = 0$). Although this fit minimizes the importance-weighted empirical risk, it exhibits severe oscillations, especially outside the region where training data are dense. This behavior reflects the high variance induced by importance weights: in our experiment, the estimated logistic DRE weights have mean 0.592 with a wide range (min = 0.047, max = 4.257), so a small number of highly upweighted, test-typical but training-rare points can dominate the objective. This need for regularization is amplified by the fact that we use a degree-10 polynomial model with only $n_{\text{tr}} = 50$ training samples, a setting in which the unregularized estimator is inherently high-variance and susceptible to overfitting. Consequently, unregularized IWERM performs very poorly in terms of test MSE (99.29), even worse than unweighted ERM (56.37),(not shown on panel (a)).

The green curve shows regularized IWERM with λ chosen by IWCV ($\lambda = 1.371$). The regularized solution is markedly smoother and better behaved, particularly in the test region. Visually, it tracks the underlying trend of f^* more faithfully where test points concentrate, suggesting improved local generalization around the testing region compared to the unregularized fit. This is also reflected quantitatively: regularized IWERM with the IWCV-selected λ achieves a much lower test MSE (0.3418) than both unregularized IWERM (test MSE of 99.2946) and regularized IWERM with the CV-selected λ (0.5044).

Panel (b) explains why the optimal choice of λ depends on the validation criterion. Standard CV (blue curve) evaluates validation error under the training distribution p_{tr} and therefore selects a very small λ ($\lambda_{\text{CV}} = 0.01001$), favoring a highly flexible model that fits the training-dense region well (CV error = 0.2743). However, this leads to poorer test performance under covariate shift. In contrast, IWCV (orange curve) reweights validation errors to approximate the test risk and selects a much larger λ ($\lambda_{\text{IWCV}} = 1.371$, IWCV error = 0.1854), which appropriately counteracts the variance introduced by importance weighting. This choice yields a smoother function, better test performance, and more stable behavior in the test region.

5 Conclusion

Covariate shift breaks the standard guarantee that minimizing training risk leads to low test error: ERM estimates R_{tr}, not R_{te}. Importance sampling yields a direct fix by expressing the test risk as a weighted train-

ing expectation, leading to IWERM. Since the weights $p_{\text{te}}(x)/p_{\text{tr}}(x)$ are unknown, DRE provides a practical route to estimate them from unlabeled samples. Finally, hyperparameter selection must also be adapted: standard cross-validation tunes for the training distribution, while IWCV reweights validation error to better reflect the test objective. Overall, the pipeline

$$\text{DRE} \rightarrow \text{IWERM} \rightarrow \text{IWCV}$$

gives a coherent approach to regression under covariate shift. Remains the matter of the practical application of these concepts. This whole project was realized on a synthetic data set. This, of course, raises the question of the modalities of the application of this theory on a real noisy and high dimensional data set (e.g. spam detection or financial data).

In more extreme settings, the discrepancy between p_{tr} and p_{te} could be so large that importance sampling becomes unstable or ineffective in practice, for instance due to highly variable or unbounded density ratios. Understanding the limitations of importance weighting under severe covariate shift, and developing robust alternatives in such regimes, raises interesting questions for future work.

A Implementation Details

In this project, we focused primarily on exploring how the existing theory on covariate shift applies to a specific problem. Our approach placed particular emphasis on understanding how and why regularization of the model (via a ridge penalty on the parameters) is effective in such settings. For the regression and classification components, we used standard implementations from *scikit-learn* (`sklearn.linear_model.LogisticRegression` and `sklearn.kernel_approximation.Nystroem`), while the density-ratio estimation (DRE) procedure itself was implemented by us.

Data generation. Synthetic data were generated under covariate shift with $f^*(x) = \sin(2\pi x)$, training inputs $x^{\text{tr}} \sim \mathcal{N}(0.75, 0.3^2)$, test inputs $x^{\text{te}} \sim \mathcal{N}(1.25, 0.1^2)$, and additive noise $\varepsilon \sim \mathcal{N}(0, 0.4^2)$. For reference, we also computed the *analytic* importance weights $w_{\text{exact}}(x) = p_{\text{te}}(x)/p_{\text{tr}}(x)$ using the Gaussian densities; these were used only as a diagnostic and not in the IWERM fits reported in the main text.

Polynomial regression and (regularized) IWERM. Polynomial regression was implemented explicitly via the design matrix

$$\Phi_M(x) = \big(1, x, x^2, \ldots, x^M\big), \qquad f_\theta(x) = \Phi_M(x)^\top \theta.$$

Importance-weighted empirical risk minimization (IWERM) with ridge regularization was implemented by rewriting the weighted objective as an ordinary least-squares problem on the rescaled data $(\sqrt{w_i}\Phi_M(x_i), \sqrt{w_i}y_i)$. For $\lambda = 0$ we solved the weighted least-squares problem using `numpy.linalg.lstsq`, and for $\lambda > 0$ we solved the regularized normal equations $(X_w^\top X_w + \lambda I)\theta = X_w^\top y_w$ using `numpy.linalg.solve`.

Density-ratio estimation via kernel logistic regression. To estimate the density ratio $w(x) = p_{\text{te}}(x)/p_{\text{tr}}(x)$ from samples, we used a classification-based method. Concretely, we trained an ℓ_2-regularized logistic regression classifier on Nyström features, where the Nyström map provides a finite-dimensional approximation of an RBF kernel feature space. Denoting $\hat{p}(y = 1 \mid x)$ the predicted probability that x is a test point, we compute

$$\hat{w}(x) = \frac{\hat{p}_{\text{te}}(x)}{\hat{p}_{\text{tr}}(x)} = \frac{\hat{p}(y = 1 \mid x)}{\hat{p}(y = 0 \mid x)} \cdot \frac{n_{\text{tr}}}{n_{\text{te}}}, \qquad \hat{p}(y = 0 \mid x) = 1 - \hat{p}(y = 1 \mid x),$$

and evaluate $\hat{w}(x)$ on the training inputs to obtain weights used in IWERM and IWCV. The conversion from classifier probabilities to the ratio estimator is implemented in our `LogisticDRE` class:

```python
import numpy as np

class LogisticDRE:
    """Density-Ratio Estimator using Logistic Regression."""
    def __init__(self, nystroem, logistic_model, n_tr, n_te):
        self.nystroem = nystroem
        self.model = logistic_model
        self.n_tr = n_tr    # number of train (denominator) samples
        self.n_te = n_te    # number of test (numerator) samples

    def predict_ratio(self, x):
        """Compute r(x) = p_te(x) / p_tr(x)."""
        x = np.asarray(x)
        if x.ndim == 1:
            x = x.reshape(-1, 1)

        z = self.nystroem.transform(x)
        p_test = self.model.predict_proba(z)[:, 1]   # y=1 = test
        p_train = 1.0 - p_test
        ratio = (p_test / np.clip(p_train, 1e-12, None)) * (self.n_tr / self.n_te)
        return ratio
```

Hyperparameter tuning (CV and IWCV). The ridge parameter λ was selected by grid search over a fixed λ-grid. Standard K-fold cross-validation (CV) reports the *unweighted* validation mean-squared error on each validation fold, whereas importance-weighted cross-validation (IWCV) reports the validation mean-squared error multiplied by $\hat{w}(x)$ and averaged over folds. We used $K = 5$ folds throughout.